Opgave 2g

Christer K. Lindholm, Carl-Emil Rohde og Emil Olesen

September 2022

1 Opgave 2g0

1.1 Opsummering af problemformulering:

Vi skal skrive et program i Fsharp, der har til formål at kunne addere, multiplicere samt rotere vektorer, skrevet som decimaltal. Samtidigt skal koden for dette beksrives med udvalgte steps fra "The 8-step guide"

1.2 Løsningsmuligheder - Overvejelser og Argumenter:

Ud fra principperne i "8-step guiden", har vi gjort os følgende overvejelser i udarbejdelsen af delopgaverne a,b og c, som vi nu vil dukumentere for i følgende afsnit:

1.3 opg a)

Først bestemmer vi typen vi arbejeder med, som er vektorer skrevet som floats (decimaltal).

Vi bestemmer en funktion, "add" som er vores vektor1 og 2, der er en vektor og skal blive til én vektor.

Vi definere en 3. vektor, som er vektor 1 2 lagt sammen efter overstående formular. Se figur 1. Figur 5

```
(* *)
let add (vetype vec = float * float

let vec1 = vec (1.0, 2.0)
let vec2 = vec (3.0, 4.0)
c1: vec) (vec2: vec) : vec =
    vec(fst vec1 + fst vec2, snd vec1 + snd vec2)

let vec3 = add vec1 vec2
printfn "%A" vec3
```

Figure 1

optwidth
opt

```
(* *)
let vec4 = vec (3.0, 5.0)
let c = 6.0

let mul (vec4: vec) (c: float): vec =
    vec(fst vec4 * c, snd vec4 * c)

let vec5 = mul vec4 c
printfn "%A" vec5
```

Figure 2

1.4 opg b)

Vi skal gange en vektor med en konstant, og vi definerer derfor en konstant c og en ny vektor 4

Vi definerer mul som vores funktion, der skal tage en vektor, og en konstant c som en float, gange de to, og derefter give os resultatet i form af en ny vektor

Vi definerer en ny vektor, som tager resultatet fra mul funktionen og definerer som en ny vektor, efter overstående formular. Se figur 2.

Figur 5

1.5 opg c)

Vi skal rotere en vektor, som vi kalder for vektor6. Vi vil gerne definerer vores variabel a som pi. Det er denne variabel der bestemmer hvor meget vektoren skal rotere

Vi definerer cos og sin, så det giver et mere overskueligt funktionsudtryk længere fremme i processen, dernæst definerer vi en funktion rot som, er vektor6 der er defineret som en vektor, og skal blive til en vektor.

Rotation af en vektor er defineret som nedestående i figur 3, hvor x og y er defineret som vektor6' første og andenkordinat. Dernæst printer vi resultatet af dette. Se figur 3.

Figur 5

optwidth
opt

```
(* *)
let vec6 = vec (4.0, 7.0)

let a = System.Math.PI

let cos = System.Math.Cos(a)
let sin = System.Math.Sin(a)

let rot (vec6: vec): vec =
    let x = fst vec6
    let y = snd vec6

    vec(x * cos - y * sin, x * sin + y * cos)

let vec7 = rot vec6
printfn "%A" vec7
```

Figure 3

```
(* *)
let add (vec1: vec) (vec2: vec) : vec =
    vec(fst vec1 + fst vec2, snd vec1 + snd vec2)

let additionVector1 = (350.0, 350.0)
let additionVector2 = (350.0, 0.0)

let setVector (can: canvas) (col: color) (p: vec) (v: vec) : unit =
    setLine can col (toInt p) (toInt (add p v))
```

Figure 4

2 Opgave 2g1

2.1 Opsummering af problemformulering:

Vi skal ved brug af canvas tegne vektorer og lave floats om til ints, som skal resultere i 36 tegnede vektorer, som danner et hjul-ligende mønster.

2.2 (a)

Opgave (a) har vi løst ved at skrive funktionen *toInt*, der tager en vektortype som argumenter og returnerer en type af to *ints*. Omdannelse sker med den indbyggede funktion *int()*, som kaldes på første og anden parameter (med *fst* og *snd*). Se figur 6.

2.3 (b)

Denne opgaves løses med funktionen *add*, der tager to vektorer som argumenter og returnerer en vektor, hvis koordinatsæt er bestemt af de to førstekoordinater lagt sammen og de to andenkoordinator lagt sammen. I kodeudklippet i figur **??**. Vi bruger funktionen setVector til at kalde vores canvas samt de to vektorer, som skal adderes og tegnes i blå farve.

Figur 5

2.4 (c)

For at tegne 36 vektorer, vælger vi at bygge en rekursiv funktion, der kaldes 36 gange og tæller ned til 0. Figur 7 dokumenterer resultatet. I figur **??** ses koden, der løser opgaven. Vores rekursive funktion definerer vi en vinkelgrad

optwidth
opt

```
(* *)
let rot (v: vec) (a: float) : vec =
    let x = fst v
    let y = snd v

    (x*cos a - y*sin a, x*sin a + y*cos a)

let draw (w: int) (h: int) =
    let c = create w h
    let v1 = vec(350.0, 350.0)
    let v2 = vec(350.0, 0.0)
    let degree = ((2.0 * System.Math.PI)/36.0)
    let rotvec = rot v1 degree
    let vEnd = add v1 v2
    for i = 0 to 35 do
        setVector c red v1 (rot(vEnd)((float i)/(degree)))
    c

show (draw 700 700) "p to p+v"
```

Figure 5

baseret på cirklens 360 grader divideret med antallet af vinkler: $360/36 \cdot \pi/36$.
Funktionen *rot* udnytter den oplyste formel for vinkelrotationer med cosinus og sinus, som kaldes på vores definerede startvektor. Vi definerer så hver vektors endepunkt *vEnd* ved at kalde *rot* på start vektorens koordinator. Til sidste kalder den rekursive funktion vores *setVector*-funktion med de nu roterende koordinator som argumenter, hver gang det rekursive parameter n tæller 1 ned.

Figur 5

Figure 6

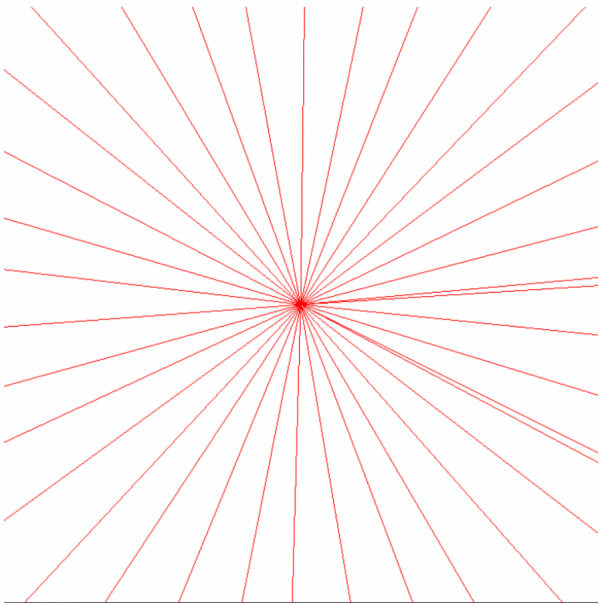


Figure 7